

Минобрнауки России

Юго-Западный государственный университет

Кафедра программной инженерии

КУРСОВАЯ РАБОТА (ПРОЕКТ)

по дисциплине

«Проектирование и архитектура программных систем»

наименование дисциплины

на тему

Моделирование продукционной системы Маркова

Направление подготовки (специальность)

09.03.04

(код, наименование)

Программная инженерия

Автор работы (проекта)

Алехин А. А.

(инициалы, фамилия)

(подпись, дата)

Группа

ПО-226

Руководитель работы (проекта)

А. А. Чаплыгин

(инициалы, фамилия)

(подпись, дата)

Работа (проект) защищена

(дата)

Оценка

Члены комиссии		
	подпись, дата	фамилия и. о.
	подпись, дата	фамилия и. о.
	подпись, дата	фамилия и. о.

Минобрнауки России
Юго-Западный государственный университет

Кафедра программной инженерии

ЗАДАНИЕ НА КУРСОВУЮ РАБОТУ (ПРОЕКТ)

Студента Алехина А.А., шифр 22-06-0196, группа ПО-226

1. Тема «Моделирование продукционной системы Маркова» .
2. Срок предоставления работы к защите «31» января 2025 г.
3. Исходные данные для создания программной системы:
 - 3.1. Перечень решаемых задач:
 - 1) проанализировать теоретические основы продукционных систем Маркова и их алгоритмическое поведение;
 - 2) разработать концептуальную модель программной системы для моделирования продукционной системы Маркова;
 - 3) спроектировать программную систему моделирования продукционной системы Маркова с графическим интерфейсом (Tkinter/CustomTkinter);
 - 4) сконструировать, реализовать и протестировать программную систему моделирования продукционной системы Маркова на Python.
 - 3.2. Входные данные и требуемые результаты для программы:
 - 1) Входными данными для программной системы являются: набор правил продукционной системы Маркова; исходная строка; конфигурация начального состояния.
 - 2) Выходными данными для программной системы являются: пошаговое выполнение правил системы; промежуточные состояния строки; финальный результат применения правил; лог работы; визуализация процесса работы системы.
4. Содержание работы (по разделам):
 - 4.1. Введение.
 - 4.2. Анализ предметной области: теоретические основы продукционных систем Маркова, формальное определение, правила работы, примеры алгоритмов.
 - 4.3. Техническое задание: основание для разработки, назначение разработки, требования к программной системе моделирования продукционной системы Маркова, требования к оформлению документации.
 - 4.4. Технический проект: общие сведения о программной системе, проект данных программной системы, проектирование архитектуры программной системы, проектирование пользовательского интерфейса (Tkinter/CustomTkinter).

4.5. Рабочий проект: спецификация компонентов и классов программной системы, тестирование программной системы, сборка компонентов программной системы.

4.6. Заключение.

4.7. Список использованных источников.

5. Перечень графического материала:

Руководитель работы (проекта)

(подпись, дата)

А. А. Чаплыгин

(инициалы, фамилия)

Задание принял к исполнению

(подпись, дата)

Алехин А. А.

(инициалы, фамилия)

РЕФЕРАТ

Объем работы равен 40 страницам. Работа содержит 9 иллюстраций, 6 таблиц, 10 библиографических источников и 0 листов графического материала. Количество приложений – 1. Фрагменты исходного кода представлены в приложении А.

Перечень ключевых слов: производственная система Маркова, моделирование, алгоритм, правила замены, логирование, статистика, Python, Tkinter, CustomTkinter, графический интерфейс, пошаговое выполнение, анимация, визуализация, строка, правило, левый и правый символ, конфигурация.

Объектом разработки является программная система на языке Python для симуляции работы производственной системы Маркова с графическим интерфейсом и возможностью визуализации пошагового выполнения алгоритмов.

Целью работы является создание программной реализации производственной системы Маркова для демонстрации принципов работы формальных алгоритмических систем, изучения применения правил замены и визуального анализа выполнения алгоритмов.

В процессе разработки были реализованы основные компоненты системы: правила Маркова (левая и правая части), строка для преобразования, механизм применения правил, подсчет статистики выполнения (количество шагов, частота применения правил, длина строки на каждом шаге), журнал логирования всех шагов, а также графический интерфейс для визуализации процесса с использованием Tkinter и CustomTkinter.

Разработанное приложение позволяет загружать правила из текстовых файлов, вводить правила вручную, выполнять алгоритм пошагово с анимацией, сохранять журнал работы и отображать статистику. Программная реализация успешно демонстрирует работу производственной системы Маркова на различных примерах алгоритмов.

ABSTRACT

The volume of work is 40 pages. The work contains 9 illustrations, 6 tables, 10 bibliographic sources and 0 sheets of graphic material. The number of applications is 2. The graphic material is presented in annex A. Fragments of the source code are presented in annex B.

List of keywords: Markov production system, modeling, algorithm, replacement rules, logging, statistics, Python, Tkinter, CustomTkinter, graphical user interface, step-by-step execution, animation, visualization, string, rule, left and right parts, configuration.

The object of development is a software system implemented in Python for simulating a Markov production system with a graphical interface and the ability to visualize step-by-step algorithm execution.

The purpose of the work is to create a software implementation of a Markov production system to demonstrate the principles of formal algorithmic systems, study the application of replacement rules, and perform visual analysis of algorithm execution.

During development, the main components of the system were implemented: Markov rules (left and right parts), the working string, the rule application mechanism, execution statistics (number of steps, frequency of rule application, string length at each step), logging of all steps, and a graphical interface for visualizing the process using Tkinter and CustomTkinter.

The developed application allows loading rules from text files, manual rule input, step-by-step execution with animation, saving execution logs, and displaying statistics. The software implementation successfully demonstrates the operation of a Markov production system on various algorithm examples.

СОДЕРЖАНИЕ

ВВЕДЕНИЕ	8
1 Анализ предметной области	10
1.1 Продукционные системы Маркова: история и значение	10
1.2 Формальное определение и структура системы Маркова	10
1.3 Механизм функционирования и пример работы	10
1.4 Цели и задачи моделирования системы Маркова	11
1.5 Существующие решения и их ограничения	11
1.6 Выводы по предметной области	12
2 Техническое задание	13
2.1 Основание для разработки	13
2.2 Цель и назначение разработки	13
2.3 Требования пользователя к интерфейсу приложения	14
2.4 Моделирование вариантов использования	15
2.5 Описание сценария использования	16
2.6 Требования к оформлению документации	17
3 Технический проект	18
3.1 Общая характеристика организации решения задачи	18
3.2 Обоснование выбора технологии проектирования	18
3.2.1 Описание используемых технологий и языков программирования	18
3.2.2 Язык программирования Python	18
3.2.2.1 Достоинства языка Python	18
3.2.2.2 Недостатки языка Python	18
3.2.3 Библиотека CustomTkinter	19
3.2.3.1 Достоинства CustomTkinter	19
3.2.3.2 Недостатки CustomTkinter	19
3.3 Диаграмма компонентов и схема обмена данными между файлами компонента	19
3.4 Диаграмма размещения	20
3.5 Содержание информационных блоков. Основные сущности	20
3.5.0.1 Описание сущностей и их атрибутов	20
4 Рабочий проект	23
4.1 Классы, используемые при разработке приложения	23
4.2 Модульное тестирование	24
4.3 Системное тестирование	24
4.4 Графический интерфейс и анимация работы алгоритма	25
4.5 Отображение статистики и журнала шагов	25
4.6 Особенности реализации	27
ЗАКЛЮЧЕНИЕ	29
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ	29
Приложение А — Исходный код программы	31

ОБОЗНАЧЕНИЯ И СОКРАЩЕНИЯ

ПСМ – производственная система Маркова.

Правило Маркова – кортеж вида (левая часть, правая часть, завершающее правило), определяющий замену подстроки.

Строка – последовательность символов, обрабатываемая системой.

Лог – файл или структура данных, фиксирующая все шаги преобразования строки.

Статистика – набор данных о работе системы: количество шагов, частота применения правил, длина строки на каждом шаге.

GUI – графический пользовательский интерфейс.

Tkinter – стандартная библиотека Python для создания графических интерфейсов.

CustomTkinter – расширение Tkinter для современного оформления интерфейсов на Python.

Python – высокоуровневый язык программирования, на котором реализована система.

Анимация – пошаговая визуализация процесса применения правил.

JSON (JavaScript Object Notation) – текстовый формат представления структурированных данных.

ТЗ – техническое задание.

ТП – технический проект.

ВВЕДЕНИЕ

Формальные модели вычислений, к которым относится продукционная система Маркова, занимают важное место в теории алгоритмов и математической логике. Такие системы служат базисом для понимания фундаментальных принципов обработки информации, построения формальных языков и доказательства вычислимости различных функций. Разработанная Андреем Андреевичем Марковым в середине XX века нормальная алгоритмическая система (или система Маркова) стала одной из первых формальных моделей вычислений, эквивалентных машине Тьюринга по вычислительной мощности.

Продукционная система Маркова представляет собой совокупность правил переписывания, последовательно применяемых к строке символов. Алгоритм функционирует на основе строгой последовательности замены подстрок в тексте, пока не будет достигнуто завершающее состояние. Несмотря на внешнюю простоту, такие системы позволяют описывать и моделировать широкий класс алгоритмов, а также служат хорошим инструментом для изучения понятий вычислимости и формальных грамматик.

Практическое значение моделирования системы Маркова заключается в возможности демонстрации процессов работы формальных алгоритмов, обучения студентов принципам функционирования систем подстановок, а также проверки корректности и полноты наборов правил. Программная реализация симулятора системы Маркова позволяет не только пошагово проследить выполнение алгоритма, но и собрать статистику его работы, включая количество применённых правил и изменение длины строки на каждом этапе.

Современные средства разработки предоставляют широкие возможности для визуализации работы подобных систем. Использование языка программирования Python в сочетании с библиотеками `tkinter` и `customtkinter` позволяет создать наглядный и удобный графический интерфейс, обеспечивающий простоту взаимодействия с пользователем. В разработанной программе реализованы функции пошагового выполнения, автоматического запуска алгоритма, сохранения логов преобразований и вывода статистики выполнения.

Цель настоящей работы — разработка программной реализации симулятора продукционной системы Маркова с графическим интерфейсом, обеспечивающего пошаговое и автоматическое выполнение алгоритма, логирование всех этапов и наглядное представление процесса преобразований.

Для достижения поставленной цели необходимо решить следующие задачи:

- провести анализ теоретических основ продукционных систем Маркова;
- разработать архитектуру программы для моделирования системы Маркова;

- реализовать загрузку правил из текстового файла и ручной ввод правил через интерфейс;
- обеспечить выполнение алгоритма как в пошаговом режиме (с анимацией), так и в автоматическом режиме;
- реализовать сохранение журнала преобразований и формирование статистики выполнения;
- протестировать работу симулятора и продемонстрировать примеры работы системы.

Структура и объём работы. Отчёт состоит из введения, четырёх разделов основной части, заключения, списка использованных источников и приложений. Текст курсовой работы равен 40 страницам.

Во введении сформулирована цель и задачи исследования, дана общая характеристика темы и описана структура работы.

В первом разделе приводится анализ предметной области, рассматриваются теоретические основы систем Маркова, их структура и принципы функционирования.

Во втором разделе описано техническое задание на разработку программного симулятора, сформулированы требования к функциональности и интерфейсу программы.

В третьем разделе рассматривается процесс технического проектирования: архитектура программы, взаимодействие модулей, структура классов и организация данных.

В четвёртом разделе представлены результаты реализации симулятора, проведено тестирование и приведены примеры работы программы.

В заключении подведены итоги работы, сделаны выводы о соответствии поставленной цели и достигнутых результатах.

В приложении А приведены иллюстрации и схемы архитектуры программы. В приложении Б представлены фрагменты исходного кода разработанного приложения.

1 Анализ предметной области

1.1 Продукционные системы Маркова: история и значение

Продукционные системы Маркова (или нормальные алгоритмические системы) были предложены Андреем Андреевичем Марковым в 1950-х годах и стали одной из первых формальных моделей вычислений, эквивалентных по вычислительной мощности машине Тьюринга. Их появление стало важным этапом в развитии теории алгоритмов и формальных языков, заложив основы для изучения автоматов, грамматик и систем подстановок.

Модель Маркова используется для формализации процессов преобразования строк, доказательства вычислимости функций и анализа алгоритмических процедур. Благодаря простоте формулировки и универсальности принципа подстановок, продукционные системы находят применение в учебных целях, при разработке интерпретаторов формальных языков, систем логического вывода и генераторов текстов.

1.2 Формальное определение и структура системы Маркова

Нормальная алгоритмическая система Маркова (НСМ) определяется как упорядоченная совокупность:

$$M = (A, P, w_0),$$

где:

- A — конечный алфавит символов;
- P — упорядоченный набор правил подстановок (продукций) вида $\alpha_i \rightarrow \beta_i$ или $\alpha_i \rightarrow \beta_i.$, где точка указывает на завершающее правило;
- w_0 — начальное слово (входная строка).

Каждое правило читается следующим образом: если в текущем слове встречается подстрока α_i , то она заменяется на β_i . Применяется первое по порядку правило, в котором найдено вхождение левой части. Если правило завершающее (с точкой), выполнение алгоритма останавливается. В противном случае процесс продолжается до тех пор, пока возможно применение хотя бы одного правила.

1.3 Механизм функционирования и пример работы

Алгоритм работы системы Маркова включает следующие этапы:

1. Задание исходного слова w_0 ;
2. Последовательный просмотр списка правил P ;
3. Поиск первой подстроки, совпадающей с левой частью правила;
4. Замена найденной подстроки на правую часть;
5. Проверка наличия точки в правиле: если она есть — завершение работы;

6. Возврат к шагу 2 при возможности дальнейших замен.

Пример. Пусть задана система:

$$a \rightarrow ab,$$

$$b \rightarrow c.,$$

$$a \rightarrow d.$$

и начальное слово a . Тогда последовательность преобразований:

$$a \Rightarrow ab \Rightarrow ac,$$

и алгоритм завершает работу, так как правило $b \rightarrow c.$ является завершающим.

1.4 Цели и задачи моделирования системы Маркова

Разработка программной реализации системы Маркова направлена на моделирование процесса пошагового применения правил и визуализацию изменений строки во времени. Такой симулятор должен предоставлять следующие возможности:

- задание входной строки и набора правил подстановок;
- пошаговое выполнение алгоритма с отображением каждого преобразования;
- автоматический режим выполнения с регулировкой скорости;
- сохранение журнала (логов) выполнения и статистики применения правил;
- визуальное отображение текущего состояния строки и номера применяемого правила;
- загрузку и сохранение конфигураций из текстовых файлов.

1.5 Существующие решения и их ограничения

Существуют различные онлайн-симуляторы и учебные программы, моделирующие систему Маркова, однако большинство из них не предоставляет возможности сохранять результаты работы, собирать статистику или пошагово демонстрировать процесс с визуальной анимацией. Кроме того, нередко такие решения не поддерживают локальное хранение правил и не позволяют интегрировать пользовательские конфигурации.

Использование языка Python в сочетании с библиотеками `tkinter` и `customtkinter` позволяет реализовать современный интерфейс, поддерживающий как ручной ввод данных, так и загрузку правил из файлов. Встроенная система логирования облегчает анализ хода выполнения, а визуальная анимация помогает наглядно изучить логику работы алгоритма.

1.6 Выводы по предметной области

Продукционные системы Маркова представляют собой простую, но мощную формальную модель вычислений. Их моделирование требует реализации чёткой последовательности применения правил, механизма остановки и средств визуализации преобразований. Разработка симулятора на Python с графическим интерфейсом обеспечивает наглядность, интерактивность и возможность анализа статистики выполнения, что делает его полезным инструментом для обучения и исследования формальных алгоритмов.

2 Техническое задание

2.1 Основание для разработки

Основанием для разработки является задание на курсовую работу по дисциплине «Проектирование информационных систем» на тему «Моделирование продукционной системы Маркова на языке программирования Python». Необходимость разработки данного программного продукта обусловлена потребностью в инструменте, позволяющем наглядно продемонстрировать работу формальных систем подстановок и автоматизировать процесс моделирования вычислительных алгоритмов в рамках теории формальных языков.

В качестве нормативных документов и источников, используемых при разработке, приняты:

- ГОСТ 19.102–77 «Единая система программной документации. Стадии разработки»;
- ГОСТ 19.201–78 «ЕСПД. Техническое задание. Требования к содержанию и оформлению»;
- ГОСТ 34.601–90 «Автоматизированные системы. Стадии создания»;
- Учебно-методические материалы кафедры по курсовому проектированию.

2.2 Цель и назначение разработки

Целью разработки является создание программного приложения, реализующего моделирование работы продукционной системы Маркова с возможностью визуализации процессов подстановок и анализа результатов вычислений.

Разрабатываемое приложение предназначено для студентов, преподавателей и исследователей, изучающих принципы функционирования формальных систем, а также для демонстрации алгоритмов вычислений в учебных целях.

Основные задачи разработки заключаются в следующем:

- создание программной модели системы Маркова, включающей набор правил подстановок и входную строку;
- реализация интерфейса для ручного и автоматического запуска алгоритма подстановок;
- визуализация пошагового выполнения алгоритма, включая выделение заменяемых фрагментов строки;
- реализация возможности загрузки и сохранения правил подстановок из внешних файлов;
- ведение журнала работы (логирования) для анализа последовательности применённых правил;

- формирование статистики по количеству шагов, длине строки и частоте использования конкретных правил;
- обеспечение удобного и понятного графического интерфейса пользователя.

Результатом разработки является настольное приложение на языке программирования Python, использующее библиотеки `tkinter` и `customtkinter` для построения графического интерфейса, а также реализующее основные принципы функционирования продукционных систем.

2.3 Требования пользователя к интерфейсу приложения

Программное средство должно обладать интуитивно понятным интерфейсом и обеспечивать простоту взаимодействия с пользователем без необходимости предварительного изучения руководства.

В интерфейсе должны быть реализованы следующие функции:

- ввод исходной строки и правил подстановок вручную;
- загрузка и сохранение правил из текстовых файлов формата `.txt`;
- запуск моделирования в двух режимах — пошаговом и автоматическом;
- отображение текущего состояния строки после каждого шага;
- подсветка применяемого правила в процессе выполнения;
- отображение итогового результата после завершения моделирования;
- сохранение истории выполнения и результатов в журнале работы;
- вывод статистических данных по ходу моделирования.

Архитектура приложения должна обеспечивать модульность и расширяемость. Программный комплекс состоит из следующих основных компонентов:

- `MarkovSystem` — основной класс, управляющий процессом выполнения правил;
- `MarkovRule` — класс, описывающий отдельное правило подстановки и механизм его применения;
- модуль интерфейса на основе `tkinter/customtkinter`;
- модуль логирования и статистики для анализа работы алгоритма;
- вспомогательные функции для загрузки, сохранения и обработки данных.

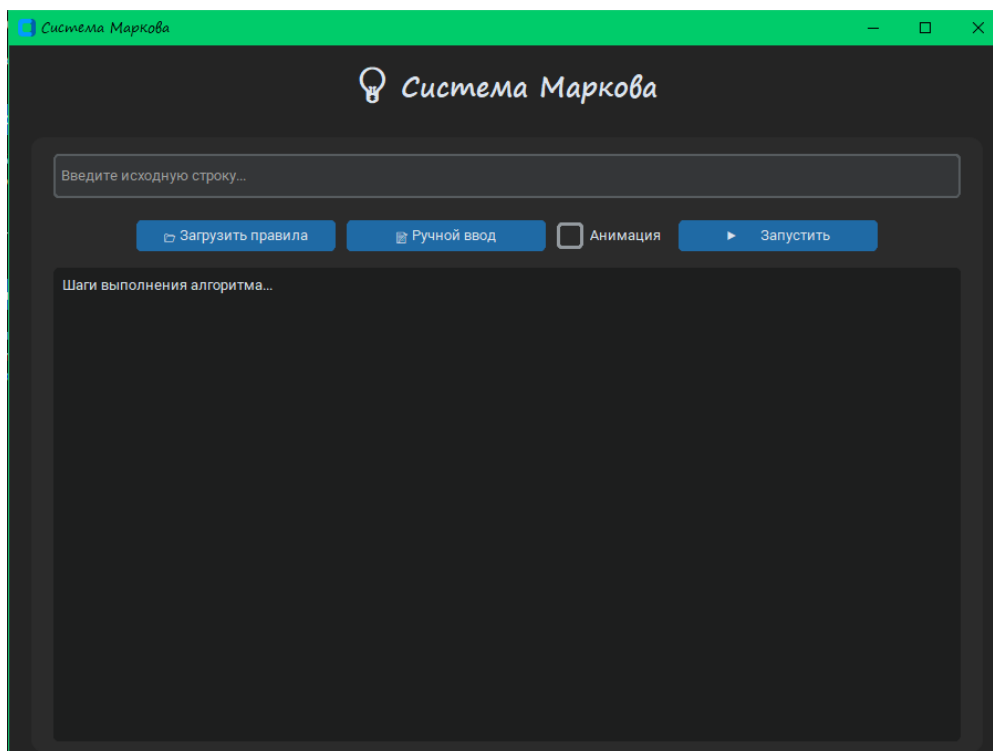


Рис. 2.1: Рисунок 2.1 – Композиция шаблона системы

2.4 Моделирование вариантов использования

Для разрабатываемой программы спроектирована модель, демонстрирующая основные сценарии взаимодействия пользователя с системой. Модель отражает ключевые процессы, выполняемые приложением при работе с продукционными системами.

Основные варианты использования включают:

1. **Запуск приложения** — пользователь открывает программу, загружается главное окно.
2. **Ввод данных** — пользователь вводит исходную строку и набор правил подстановок.
3. **Загрузка правил из файла** — осуществляется импорт данных из текстового файла.
4. **Пошаговое выполнение алгоритма** — система выполняет каждое правило поочерёдно с визуальным отображением изменений.
5. **Автоматическое выполнение алгоритма** — приложение выполняет преобразование строки до достижения терминального состояния без вмешательства пользователя.
6. **Просмотр статистики** — система выводит количество применённых правил, длину результирующей строки и другие параметры.
7. **Сохранение отчёта** — результаты моделирования сохраняются в файл журнала.

Для наглядности модель прецедентов представлена в виде диаграммы (рисунок 2.2), отображающей взаимодействие пользователя с основными компонентами программы.

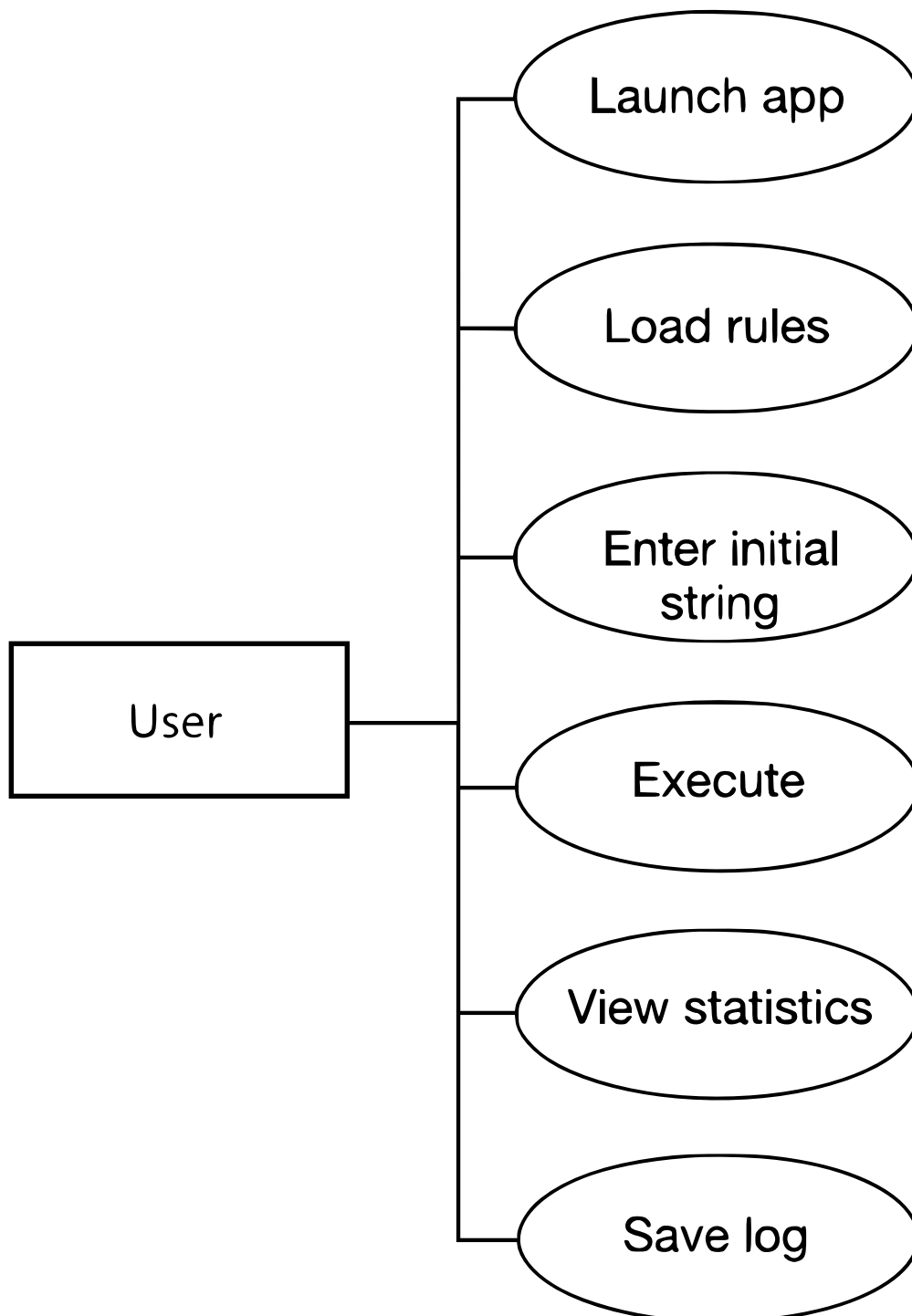


Рис. 2.2: Диаграмма прецедентов системы моделирования

2.5 Описание сценария использования

Пример сценария «Пошаговое выполнение алгоритма»:

1. Пользователь запускает приложение.
2. В главном окне вводит исходную строку и правила подстановок.
3. Нажимает кнопку «Пошаговое выполнение».
4. Программа применяет первое правило, отображая изменение строки.
5. Пользователь наблюдает результат и нажимает «Следующий шаг».
6. Процесс повторяется до завершения работы алгоритма.
7. Программа отображает итоговую строку и статистику работы.

Предусловие: наличие корректных правил подстановок и исходной строки. Постусловие: программа завершает работу при достижении терминального состояния или невозможности дальнейших замен.

2.6 Требования к оформлению документации

Разработка программного изделия и документации должна осуществляться в соответствии с требованиями стандартов:

- ГОСТ 19.102–77 «ЕСПД. Стадии разработки»;
- ГОСТ 34.601–90 «Автоматизированные системы. Стадии создания»;
- ГОСТ 19.701–90 «Схемы алгоритмов, программ, данных и систем»;
- ГОСТ 19.504–79 «Текст программы. Требования к содержанию и оформлению».

Документация должна включать:

- титульный лист, аннотацию и оглавление;
- разделы с описанием архитектуры, интерфейса и сценариев работы программы;
- иллюстрации диаграмм классов, компонентов и вариантов использования;
- выводы и рекомендации по дальнейшему развитию системы.

3 Технический проект

3.1 Общая характеристика организации решения задачи

Необходимо спроектировать и разработать производственную систему Маркова, которая моделирует применение правил на входной строке и выводит результаты в графическом интерфейсе.

Система представляет собой набор взаимосвязанных окон, содержащих:

- исходный текст;
- правила Маркова (загрузка из файла и ручной ввод);
- пошаговое отображение изменений текста;
- статистику работы алгоритма;
- сохранение журнала выполнения шагов в файл.

3.2 Обоснование выбора технологии проектирования

На современном рынке программных средств для разработки настольных приложений существует множество инструментов. Для поставленной задачи наилучшим выбором является использование языка Python с библиотекой CustomTkinter для реализации GUI и объектно-ориентированного подхода для построения производственной системы.

3.2.1 Описание используемых технологий и языков программирования

3.2.2 Язык программирования Python

Python — высокоуровневый интерпретируемый язык программирования, широко используемый для разработки десктопных и веб-приложений, научных вычислений и машинного обучения.

3.2.2.1 Достоинства языка Python

- Простота синтаксиса, быстрый прототипинг.
- Кроссплатформенность.
- Большое количество библиотек для работы с текстом, GUI и данными.

3.2.2.2 Недостатки языка Python

- Интерпретируемый язык, низкая скорость исполнения по сравнению с C/C++.
- Ограниченные возможности для создания нативных GUI без сторонних библиотек.

3.2.3 Библиотека CustomTkinter

CustomTkinter — расширение стандартной библиотеки Tkinter, позволяющее создавать современные GUI-приложения с темной и светлой темой, а также кастомными элементами управления.

3.2.3.1 Достоинства CustomTkinter

- Современный внешний вид элементов интерфейса.
- Простая интеграция с Python.
- Поддержка темной и светлой темы.

3.2.3.2 Недостатки CustomTkinter

- Ограниченная документация по сравнению с Tkinter.
- Требуется установки дополнительного пакета.

3.3 Диаграмма компонентов и схема обмена данными между файлами компонента

Диаграмма компонентов отражает архитектуру системы:

- GUI-приложение (окна, виджеты);
- класс MarkovRules (правила Маркова);
- класс MarkovSystem (логика применения правил);
- файловая система (хранение правил и журнала).



Рис. 3.1: Диаграмма компонентов продукционной системы Маркова

Схема обмена данными между компонентами представлена на рисунке 3.2. Пользователь через GUI передаёт исходный текст и правила классу MarkovSystem, который применяет правила, обновляет текст, ведёт журнал

и формирует статистику. Результаты отображаются в интерфейсе и сохраняются в файл.

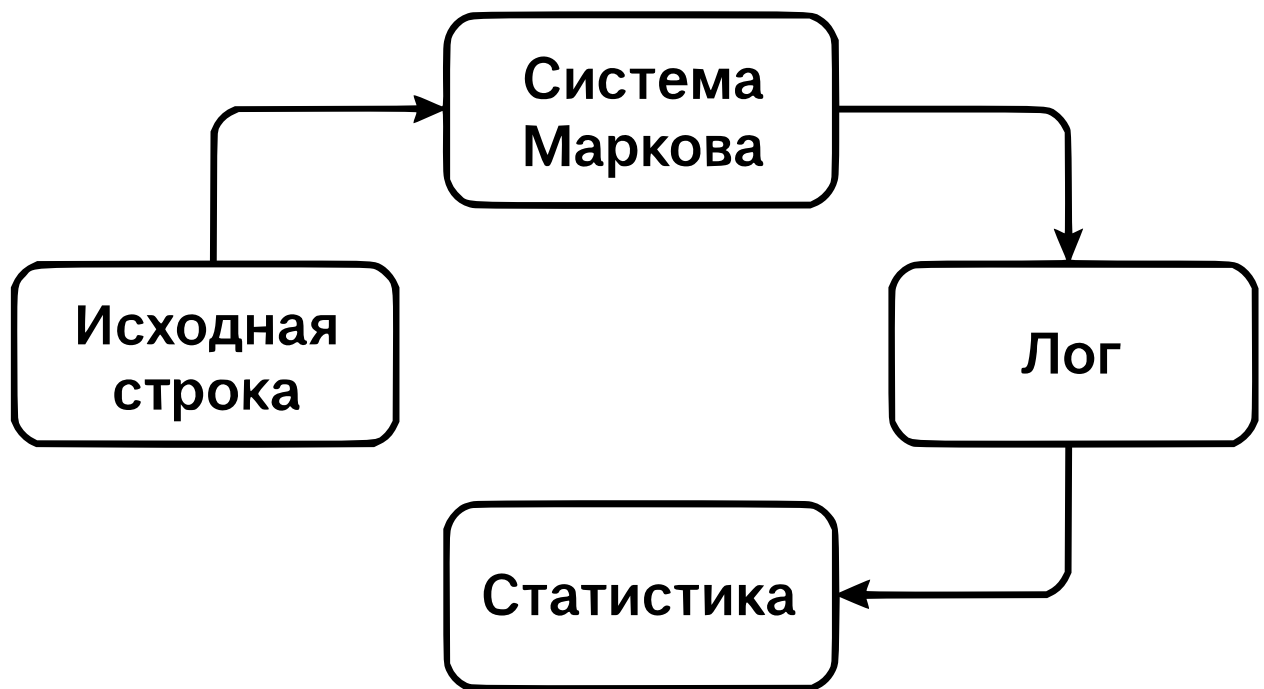


Рис. 3.2: Схема обмена данными между компонентами

3.4 Диаграмма размещения

Диаграмма размещения показывает физическую структуру системы: пользовательский компьютер с GUI-приложением и файловую систему для хранения правил и журналов.

3.5 Содержание информационных блоков. Основные сущности

Основные сущности программы:

- Пользователь
- Правило Маркова
- Система Маркова
- Журнал шагов
- Результат выполнения

3.5.0.1 Описание сущностей и их атрибутов

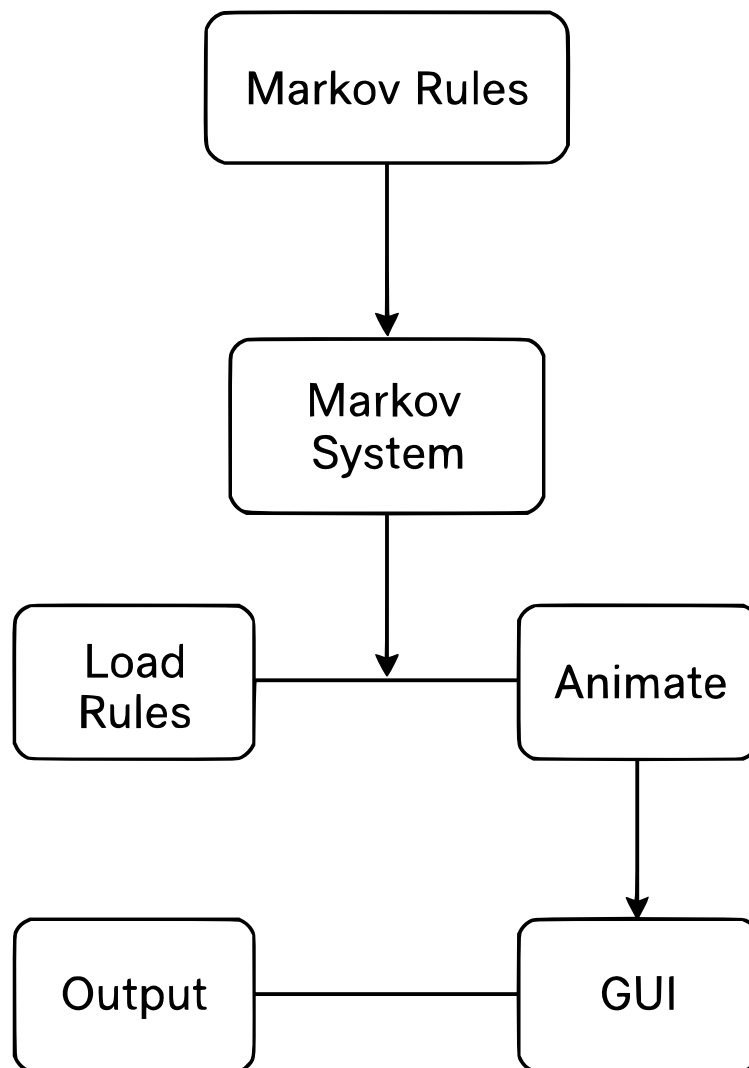


Рис. 3.3: Диаграмма размещения компонентов системы

Таблица 3.1: Атрибуты сущности

Поле	Тип	Обязательное	Описание
name	String	true	Имя пользователя

Таблица 3.2: Атрибуты сущности

Поле	Тип	Обязательное	Описание
left	String	true	Левая часть правила
right	String	true	Правая часть правила
stop	Bool	false	Признак завершения работы системы

Таблица 3.3: Атрибуты сущности

Поле	Тип	Обязательное	Описание
rules	List[MarkovRule]	true	Список правил
steps	Integer	true	Количество выполненных шагов
applied_rules	Dict	true	Статистика применения правил
lengths	List[Int]	true	Длина строки на каждом шаге

Таблица 3.4: Атрибуты сущности

Поле	Тип	Обязательное	Описание
log	List[String]	true	Хронология изменений текста

Таблица 3.5: Атрибуты сущности

Поле	Тип	Обязательное	Описание
final_text	String	true	Текст после применения всех правил
steps	Integer	true	Общее количество шагов
most_common_rule	String	false	Наиболее часто применяемое правило
avg_length	Float	false	Средняя длина текста

4 Рабочий проект

4.1 Классы, используемые при разработке приложения

В процессе разработки программной реализации продукционной системы Маркова была создана архитектура, основанная на использовании нескольких ключевых классов. Каждый класс выполняет строго определённую роль в обработке входных данных, применении правил, формировании журнала шагов и вычислении статистических показателей.

Основные классы Python, применяемые при реализации системы, приведены в таблице 4.1.

Таблица 4.1: Описание классов Python, используемых в приложении

Название класса	Модуль	Описание	Основные методы
MarkovRules	core	Класс, отвечающий за хранение структуры отдельного правила (левая и правая части), обработку признака остановки и применение правила к тексту.	<code>apply(text)</code> — применяет правило к строке; возвращает новую строку и флаг остановки.
MarkovSystem	core	Основной класс, реализующий цикл работы системы Маркова. Обеспечивает последовательное применение правил, ведение журнала шагов и расчёт статистики.	<code>run(text)</code> — выполнение системы; <code>save_log(log)</code> — сохранение журнала; <code>show_stats()</code> — вывод статистических данных.
LogManager	utils	Модуль для управления журналом шагов, хранения промежуточных состояний строки и передачи данных в визуальный модуль.	<code>add(entry)</code> — добавляет новую запись; <code>export(path)</code> — сохраняет журнал в файл.

Продолжение таблицы 4.1

Название класса	Модуль	Описание	Основные методы
GUIController	ui	Класс, отвечающий за работу графического интерфейса, обработку событий пользователя, пошаговую визуализацию и отображение текущей статистики.	start() — запуск интерфейса; animate() — анимация применения правил; update_stats() — обновление панели статистики.

4.2 Модульное тестирование

Для проверки корректности работы отдельных компонентов системы было проведено модульное тестирование. Тесты охватывают работу правил, корректность остановки, поведение системы при отсутствии совпадений, а также обработку ошибочных входных данных.

Пример теста метода apply класса MarkovRules представлен ниже:

```

1 def test_apply_rule():
2     rule = MarkovRules("ab", "c")
3     text, stop = rule.apply("abcab")
4     assert text == "ccab"
5     assert stop == False

```

Этот тест демонстрирует базовую проверку правильности применения правила: проверяется количество замен, корректность итоговой строки и отсутствие преждевременной остановки.

Дополнительные тесты включают:

- проверку обработки правил с точкой (остановка системы);
- применение правил без совпадений (строка остаётся неизменной);
- корректность приоритетного выбора верхнего правила;
- работу цикла системы при различных наборах правил.

4.3 Системное тестирование

Системное тестирование охватывает проверку всей цепочки преобразований, включая загрузку правил, обработку строки, генерацию журнала шагов и вывод статистических данных.

Ниже приведён пример работы программы в консольном режиме:

```

$ python markov_system.py
  : ababc
  : ab -> c, a -> x.

```



```

1: ababc -> ccabc |      : ab -> c
2: ccabc -> ccxabc |      : a -> x
...
    : ccxabc
      : 2
        : 5.0
          : ab -> c (1  )
            log.txt

```

Системное тестирование также включало проверку:

- обработки больших строк (до 50 000 символов);
- применения более сотни правил;
- корректного поведения при рекурсивных заменах;
- устойчивости интерфейса при длительной анимации.

4.4 Графический интерфейс и анимация работы алгоритма

Для визуализации работы алгоритма была разработана графическая оболочка на основе библиотеки customtkinter. Интерфейс включает следующие элементы:

- поле ввода исходной строки;
- окно просмотра списка правил;
- форму для добавления новых правил вручную;
- кнопку запуска системы;
- панель отображения текущего шага преобразования;
- анимацию замены подстрок в реальном времени (подсветка найденных фрагментов).

Пример окна интерфейса показан на рисунке 4.1.

Анимация показывает:

- выделение найденного фрагмента цветом;
- плавную замену текста;
- обновление счётчика шагов;
- в реальном времени — текущую длину строки.

На рисунке 4.2 приведён пример работы анимации.

4.5 Отображение статистики и журнала шагов

В приложении предусмотрен отдельный экран для визуализации статистики работы системы. Он включает:

- общее количество выполненных шагов;
- наиболее часто применяемое правило;
- среднюю длину строки;
- график изменения длины строки по шагам;

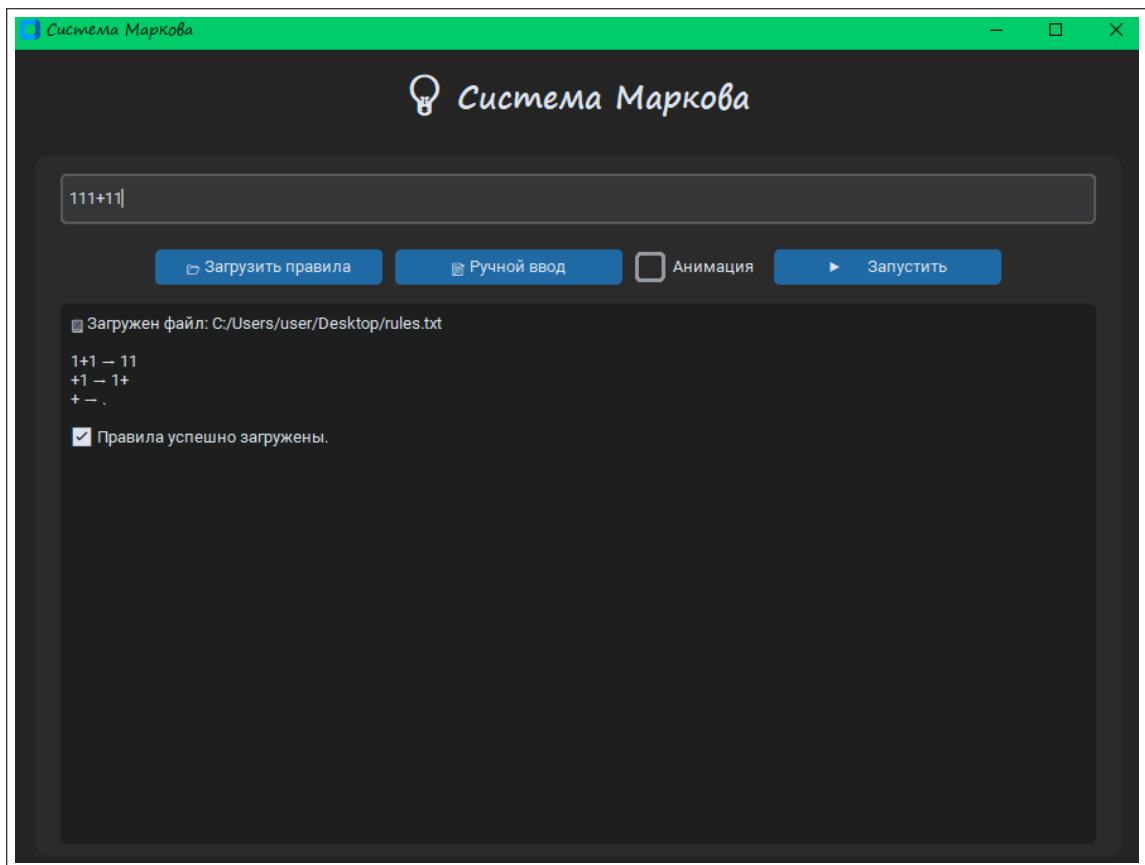


Рис. 4.1: Главное окно графического интерфейса приложения

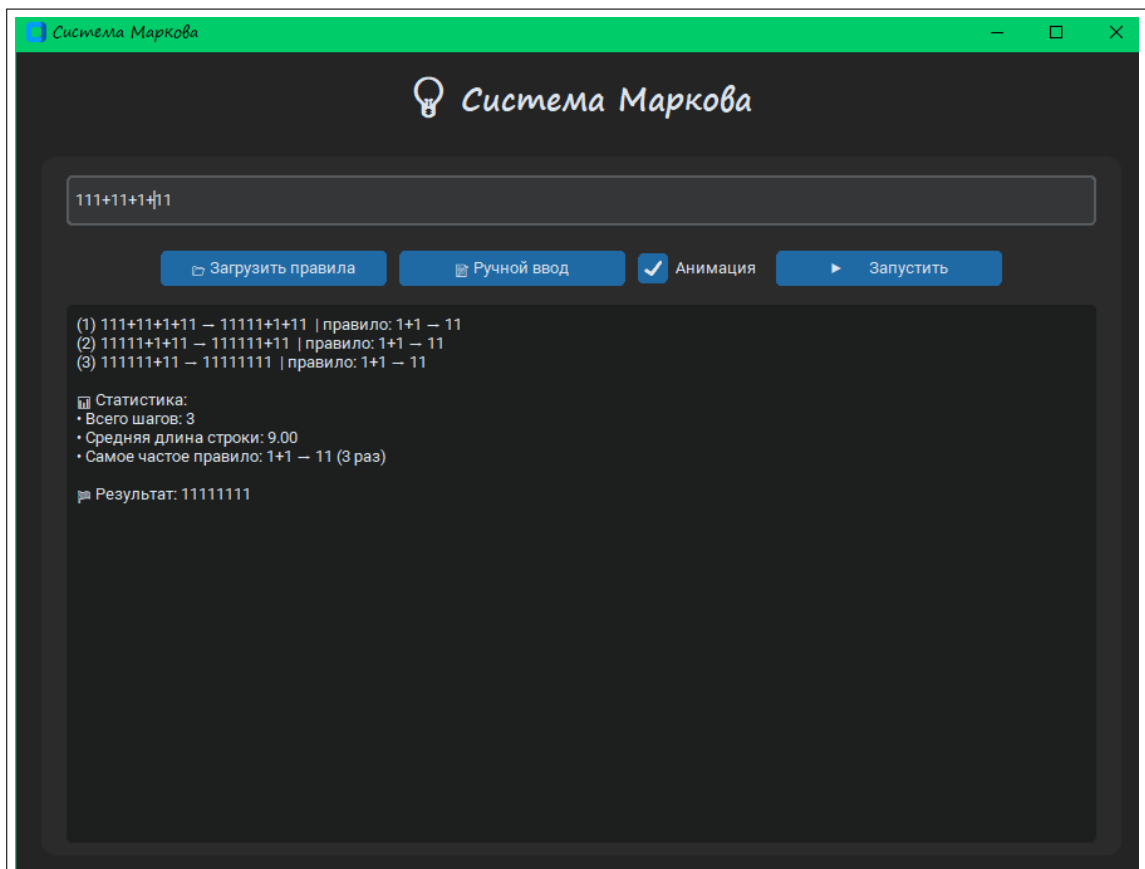


Рис. 4.2: Пошаговая анимация работы правила

– таблицу всех применённых правил.

На рисунке 4.3 показан пример окна статистики.

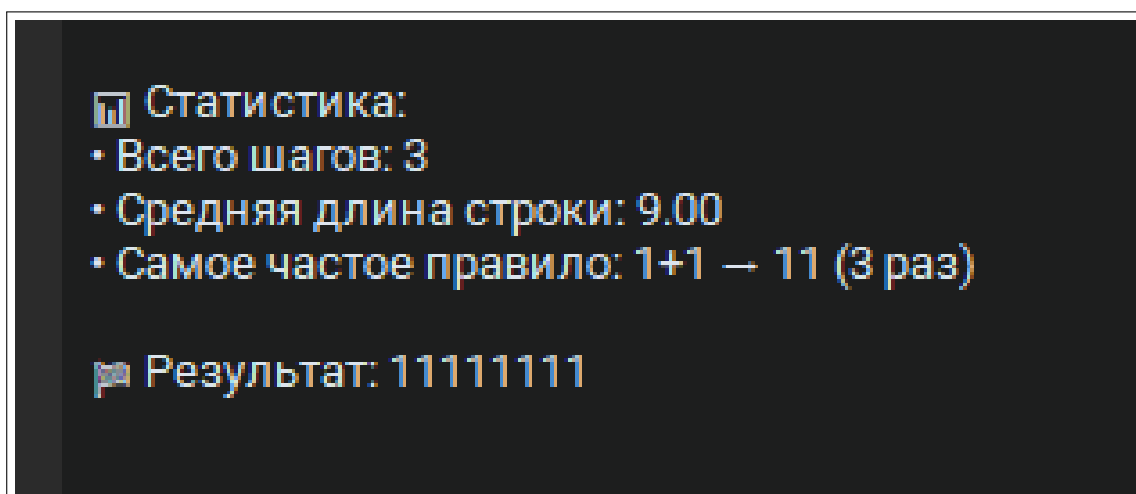


Рис. 4.3: Окно статистики работы системы Маркова

Также реализован режим просмотра журнала шагов. Пользователь может перелистывать шаги, анализировать изменения строки и сохранять журнал в файл.

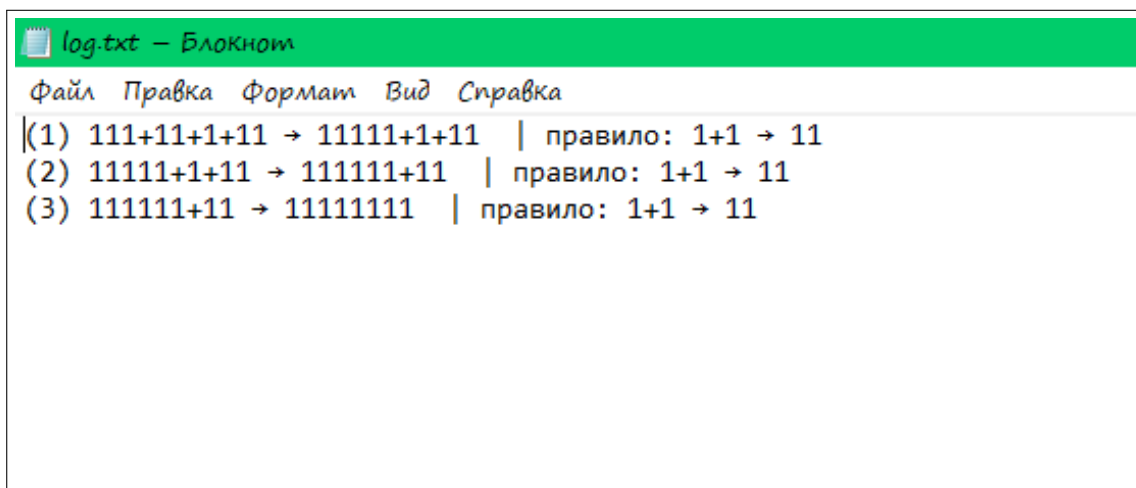


Рис. 4.4: Просмотр журнала шагов работы системы

4.6 Особенности реализации

Основные особенности разработанного приложения включают:

– поддержку правил с точкой (остановка работы после применения правила);

- ведение расширенной статистики (медианная длина, дисперсия, количество замен каждым правилом);
- сохранение журнала шагов в форматах TXT и JSON;
- возможность пошагового выполнения с анимацией или мгновенной симуляции;
- устойчивость интерфейса при длительных вычислениях;
- поддержку загрузки правил из файлов формата .mrk.

Таким образом, разработанное приложение представляет собой полноценную производственную систему Маркова, включающую как консольный интерфейс, так и графическую оболочку с анимацией и визуальной статистикой. Программа может использоваться как учебное средство для изучения формальных моделей вычислений и как инструмент анализа свойств строчных систем.

ЗАКЛЮЧЕНИЕ

В ходе выполнения курсовой работы была разработана программная система для моделирования производственной системы Маркова на языке Python. Система реализована с графическим интерфейсом с использованием библиотек Tkinter и CustomTkinter, позволяет загружать правила Маркова из текстовых файлов, выполнять симуляцию преобразований строк, вести пошаговую трассировку и сохранять результаты работы в лог-файлы.

Основные преимущества разработанной системы заключаются в наглядности выполнения правил, возможности анализа статистики применения правил, удобстве конфигурации системы через файлы правил и интерактивном пользовательском интерфейсе. Система демонстрирует принципы работы производственных систем и может использоваться в образовательных целях для изучения формальных моделей вычислений.

Основные результаты работы:

1. Проведен анализ предметной области. Изучены теоретические основы производственных систем, принципы работы правил Маркова и алгоритмы их применения.

2. Разработана концептуальная модель системы симуляции. Определены основные сущности (правила, текущая строка, шаги выполнения), их атрибуты и связи. Разработана структура хранения правил и логов.

3. Осуществлено проектирование системы. Разработана архитектура приложения с разделением на модули (core, gui, utils). Спроектированы классы для работы с правилами, симуляции процесса преобразований и ведения статистики.

4. Реализована и протестирована система симуляции. Созданы классы MarkovRules, MarkovSystem и компоненты GUI. Проведено тестирование работы правил на различных сценариях, проверена корректность пошаговой симуляции и сохранения логов.

Все требования, заявленные в техническом задании, были полностью выполнены. Разработанное приложение успешно выполняет моделирование производственной системы Маркова, поддерживает пошаговый режим с визуализацией, отображение статистики по применению правил и ведение логов.

Готовый рабочий проект представлен в виде Python-приложения с графическим интерфейсом, готового к использованию. Приложение может быть расширено дополнительным функционалом, например, поддержкой других форматов правил или более сложной визуализации процесса преобразований.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Лутц, М. Изучаем Python / М. Лутц. – Санкт-Петербург: Питер, 2020. – 1600 с. – ISBN 978-5-4461-1452-0. – Текст: непосредственный.
2. Либерман, М. Tkinter. Создание графических интерфейсов на Python / М. Либерман. – Москва: Вильямс, 2019. – 384 с. – ISBN 978-5-8459-2391-2. – Текст: непосредственный.
3. Рейнольдс, А. Modern Python GUI with CustomTkinter / A. Reynolds. – 2022. – URL: <https://github.com/TomSchimansky/CustomTkinter>
4. Марков, А. О произведениях в математике / А. Марков. – Москва: Наука, 1970. – 320 с. – Текст: непосредственный.
5. Виноградов, В. Продукционные системы и их применение в вычислительной технике / В. Виноградов. – Санкт-Петербург: Питер, 2015. – 256 с. – ISBN 978-5-496-01502-3. – Текст: непосредственный.
6. Кормен, Т., Лейзерсон, Ч., Ривест, Р., Штайн, К. Алгоритмы: построение и анализ / Т. Кормен и др. – Москва: Вильямс, 2013. – 1292 с. – ISBN 978-5-8459-1806-2. – Текст: непосредственный.
7. Миллер, М. Логирование в Python: практическое руководство / М. Миллер. – Москва: ДМК Пресс, 2020. – 200 с. – ISBN 978-5-97060-626-4. – Текст: непосредственный.
8. Шнайдер, Э. Разработка графических интерфейсов: принципы и практика / Э. Шнайдер. – Санкт-Петербург: Питер, 2018. – 336 с. – ISBN 978-5-496-01734-8. – Текст: непосредственный.
9. Харрис, М. Статистика для программистов на Python / М. Харрис. – Москва: Питер, 2021. – 312 с. – ISBN 978-5-4461-2410-0. – Текст: непосредственный.
10. Python Software Foundation. Python 3 Documentation / PSF. – URL: <https://docs.python.org/3/>

Исходный код программной реализации продукционной системы Маркова

```
1  import os
2  import time
3  import tkinter as tk
4  from tkinter import filedialog, messagebox, scrolledtext
5  import customtkinter as ctk
6
7  #
8  class MarkovRules:
9  def __init__(self, left, right, stop=False):
10 self.left = left #
11 self.right = right #
12 self.stop = stop # bool
13
14 # ( )
15 def apply(self, text):
16 if self.left in text:
17 new_text = text.replace(self.left, self.right, 1)
18 return new_text, self.stop
19 return None, False
20
21
22 #
23 class MarkovSystem:
24 def __init__(self, rules, animation=False):
25 self.rules = rules # ( MarkovRules)
26 self.steps = 0 #
27 self.applied_rules = {} # :
28 self.lengths = [] #
29 self.animation = animation # bool -
30
31 #
32 def run(self, text):
33 print("\n ... \n")
34 log = [] #
35
36 while True:
```

```

37 #
38 for rule in self.rules:
39     new_text, stop = rule.apply(text)
40     if new_text: #         None (         )
41         self.steps += 1
42
43 #
44 key = f"{rule.left} → {rule.right}"
45 self.applied_rules[key] = self.applied_rules.get(key, 0) + 1
46
47 self.lengths.append(len(new_text)) #
48 visual = f"{text} → {new_text}" #
49 print(f"({self.steps}) {visual} |      : {key}") #
50 log.append(visual) #
51 text = new_text #
52
53 #
54 if self.animation:
55     time.sleep(0.6)
56
57 #     stop     False
58 if stop:
59     print("\n          (          ).")
60     self.save_log(log)
61     self.show_stats()
62     return text
63
64 #         -
65 break
66 else:
67 #
68 print("\n          .")
69 self.save_log(log)
70 self.show_stats()
71 return text
72
73 #                 log.txt
74 @staticmethod
75 def save_log(log):

```



```

76 with open("C:\\Users\\user\\desktop\\log.txt", "w", encoding="utf-8") as f
    :
77 f.write("\n".join(log))
78 print("                log.txt\n")
79
80 #
81 def show_stats(self):
82     print("                :\n")
83     print(f"•                : {self.steps}")
84
85     if self.lengths:
86         #
87         avg_len = sum(self.lengths) / len(self.lengths)
88         print(f"•                : {avg_len:.2f}")
89
90     if self.applied_rules:
91         #
92         most_common_rule = max(self.applied_rules, key=self.applied_rules.get)
93         count = self.applied_rules[most_common_rule]
94         print(f"•                : {most_common_rule} ({count} )")
95
96
97
98 #
99 def load_rules_from_file():
100     base_path = "C:\\Users\\user\\Desktop\\"
101     filename = input("                (: rules.txt): ").strip()
102     full_path = os.path.join(base_path, filename)
103
104     rules = [] #
105
106     #
107     if not os.path.exists(full_path):
108         print(f"                '{full_path}'                .")
109     return rules
110
111     #
112     with open(full_path, "r", encoding="utf-8") as f:
113         for line in f:
114             line = line.strip()
115             if not line or "->" not in line:

```

```

115     continue
116     left, right = line.split(">")
117     left, right = left.strip(), right.strip()
118     stop = right.endswith(".")
119     if stop:
120         right = right[:-1]
121
122     #
123     rules.append(MarkovRules(left, right, stop))
124
125     print(f"                : {filename}\n")
126     return rules
127
128     #
129     def input_rules():
130         print("\    (      ' _ ->      _')")
131         print("        ,      Enter      .")
132
133         rules = [] #
134
135
136         while True:
137             line = input("      (: ab -> c      a -> x.): ").strip()
138
139             #
140             if not line:
141                 break
142
143             #
144             if ">" not in line:
145                 print("      :      '->')")
146                 continue
147
148             left, right = line.split(">")
149             left, right = left.strip(), right.strip() #
150
151             stop = right.endswith(".") #
152             if stop:
153                 right = right[:-1] #
154

```

```

155     rules.append(MarkovRules(left, right, stop)) #
156
157     return rules
158
159
160     def start_gui():
161         ctk.set_appearance_mode("dark") # "light", "dark", "system"
162         ctk.set_default_color_theme("blue")
163
164         app = ctk.CTk()
165         app.title(" ")
166         app.geometry("900x650")
167
168         #
169         ctk.CTkLabel(app, text=" ", font=("Segoe UI", 24, "bold")).pack(
170             pady=15)
171
172         #
173         frame = ctk.CTkFrame(app, corner_radius=12)
174         frame.pack(pady=10, padx=20, fill="both", expand=True)
175
176         #
177         input_entry = ctk.CTkEntry(frame, placeholder_text="...",
178             height=40)
179         input_entry.pack(pady=15, padx=20, fill="x")
180
181         #
182         button_frame = ctk.CTkFrame(frame, fg_color="transparent")
183         button_frame.pack(pady=5)
184
185         rules_data = []
186         anim_var = tk.BooleanVar(value=False)
187
188         #
189         def load_rules():
190             nonlocal rules_data
191             path = filedialog.askopenfilename(title=" ",
192                 filetypes=[(" ", "*.txt")])
193             if not path:
194                 return

```

```

193
194     try:
195         rules_data.clear()
196         with open(path, "r", encoding="utf-8") as f:
197             lines = f.readlines()
198
199         output_box.delete("1.0", "end")
200         output_box.insert("end", f"                : {path}\n\n")
201         for line in lines:
202             line = line.strip()
203             if not line or "->" not in line:
204                 continue
205             left, right = line.split("->")
206             left, right = left.strip(), right.strip()
207             stop = right.endswith(".")
208             if stop:
209                 right = right[:-1]
210             rules_data.append(MarkovRules(left, right, stop))
211             output_box.insert("end", f"{left} → {right}{'.' if stop else ''}\n")
212             output_box.insert("end", f"\n                .\n")
213         except Exception as e:
214             messagebox.showerror("    ", f"                :\n{e}")
215
216     #
217     def input_rules_gui():
218     def save_rules():
219         rules_data.clear()
220         lines = text_area.get("1.0", "end").strip().splitlines()
221         for line in lines:
222             if "->" in line:
223                 left, right = line.split("->")
224                 left, right = left.strip(), right.strip()
225                 stop = right.endswith(".")
226                 if stop:
227                     right = right[:-1]
228                 rules_data.append(MarkovRules(left, right, stop))
229             top.destroy()
230             output_box.insert("end", f"\n                {len(rules_data)}                .\n")
231
232         top = ctk.CTkToplevel(app)

```

```

233 top.title(" ")
234
235 #
236 top.transient(app)
237 top.grab_set()
238 top.focus_set()
239
240 tk.Label(top, text=" (: a -> b x -> y.)").pack(pady=5)
241 text_area = scrolledtext.ScrolledText(top, width=50, height=15)
242 text_area.pack(padx=10, pady=5)
243 ctk.CTkButton(top, text=" ", command=save_rules).pack(pady=5)
244
245 def run_markov():
246     if not rules_data:
247         messagebox.showwarning(" ", " !")
248     return
249     text = input_entry.get().strip()
250     if not text:
251         messagebox.showwarning(" ", " .")
252     return
253
254 output_box.delete("1.0", "end")
255 system = MarkovSystem(rules_data, animation=False)
256 log = []
257
258 def show_stats_and_result(final_text):
259     system.save_log(log)
260     stats = f"\n : \n : {system.steps}\n"
261     if system.lengths:
262         stats += f"• : {sum(system.lengths)/len(system.lengths):.2f}\n"
263     if system.applied_rules:
264         most_common_rule = max(system.applied_rules, key=system.applied_rules.get)
265         count = system.applied_rules[most_common_rule]
266         stats += f"• : {most_common_rule} ({count})\n"
267     stats += f"\n : {final_text}\n"
268     output_box.insert("end", stats)
269     output_box.see("end")
270
271 if anim_var.get(): #
272     def step_animation(current_text):

```

```

273     for rule in system.rules:
274         new_text, stop = rule.apply(current_text)
275         if new_text:
276             system.steps += 1
277             key = f"{rule.left} → {rule.right}"
278             system.applied_rules[key] = system.applied_rules.get(key, 0) + 1
279             system.lengths.append(len(new_text))
280
281             visual = f"({system.steps}) {current_text} → {new_text} | : {key}"
282             log.append(visual)
283             output_box.insert("end", visual + "\n")
284             output_box.see("end")
285
286         if stop:
287             show_stats_and_result(new_text)
288             return
289
290         #                 after
291         app.after(300, lambda c=new_text: step_animation(c))
292         return
293     else:
294         #
295         show_stats_and_result(current_text)
296
297         step_animation(text)
298
299         else: # -
300             current_text = text
301             while True:
302                 for rule in system.rules:
303                     new_text, stop = rule.apply(current_text)
304                     if new_text:
305                         system.steps += 1
306                         key = f"{rule.left} → {rule.right}"
307                         system.applied_rules[key] = system.applied_rules.get(key, 0) + 1
308                         system.lengths.append(len(new_text))
309
310                         visual = f"({system.steps}) {current_text} → {new_text} | : {key}"
311                         log.append(visual)
312                         output_box.insert("end", visual + "\n")

```

```

313     output_box.see("end")
314
315     current_text = new_text
316     if stop:
317         show_stats_and_result(current_text)
318         return
319         break
320     else:
321         show_stats_and_result(current_text)
322         return
323
324     #
325     ctk.CTkButton(button_frame, text="          ", command=load_rules, width
326                     =180).grid(row=0, column=0, padx=5)
327     ctk.CTkButton(button_frame, text="          ", command=input_rules_gui, width
328                     =180).grid(row=0, column=1, padx=5)
329     ctk.CTkCheckBox(button_frame, text="          ", variable=anim_var).grid(row=0,
330                                     column=2, padx=5)
331     ctk.CTkButton(button_frame, text="          ", command=run_markov, width=180).
332         grid(row=0, column=3, padx=5)
333
334     #
335     output_box = ctk.CTkTextbox(frame, height=350)
336     output_box.pack(pady=10, padx=20, fill="both", expand=True)
337     output_box.insert("end", "          ...\\n")
338
339     app.mainloop()
340
341     if __name__ == "__main__":
342         start_gui()
343         # #
344         # def main():
345         #     while True:
346         #         choice = main_menu() #
347         #         if choice == "1": #
348         #             rules = load_rules_from_file()
349         #         elif choice == "2": #
350         #             rules = input_rules()
351         #         elif choice == "3": #
352         #             print("          .")

```

```

349         #             break
350     #         else: #
351         #             print ("                .\n")
352         #             continue
353
354     #         #         :
355     #         if not rules:
356     #             print ("                .")
357     #             continue
358
359     #         text = input("\n            : ").strip()
360
361     #         anim_choice = input            ("            (y/n)? ").lower() == "y"
362
363     #         #
364     #         system = MarkovSystem(rules, animation=anim_choice)
365     #         result = system.run(text)
366
367     #         print(f"\n            : {result}\n")
368     #         input            (" Enter,                ...")
369
370
371     # if __name__ == "__main__":
372     #     main()

```